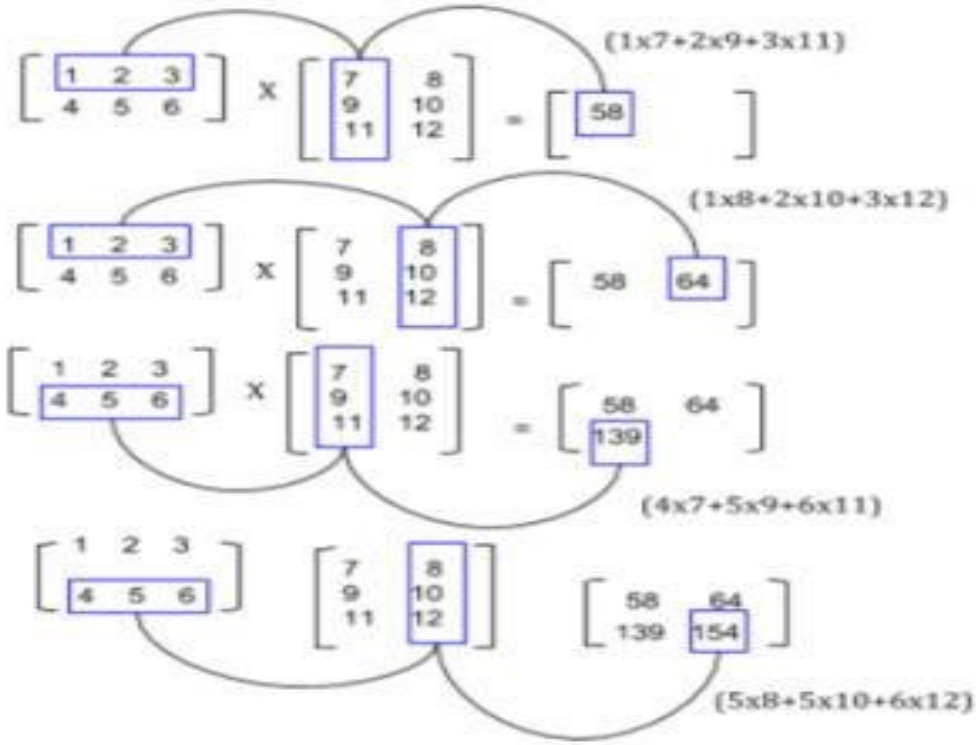
Lab Assignment No.	3
Title	Implement Matrix Multiplication using Map-Reduce
Roll No.	
Class	BE
Date of Completion	
Subject	Computer Laboratory-IV : Big Data Analytics
Assessment Marks	
Assessor's Sign	

AIM: To Develop a MapReduce program to implement Matrix Multiplication.

In **mathematics**, **matrix multiplication** or the **matrix product** is a binary operation that produces a matrix from two matrices. The definition is motivated by linear equations and linear transformations on vectors, which have numerous applications in applied mathematics, physics, and engineering. In more detail, if **A** is an $n \times m$ matrix and **B** is an $m \times p$ matrix, their matrix product **AB** is an $n \times p$ matrix, in which the m entries across a row of **A** are multiplied with the m entries down a column of **B** and summed to produce an entry of **AB**. When two linear transformations are represented by matrices, then the matrix product represents the composition of the two transformations.



Algorithm for Map Function.

a. for each element m_{ij} of M do
produce (key,value) pairs as ((i,k), (M,j,m_{ij}), for k=1,2,3,.. upto the number of columns of N

b. for each element njk of N do produce (key,value) pairs as ((i,k),(N,j,N_{jk}), for i = 1,2,3,.. Upto the number of rows of M.

c. return Set of (key,value) pairs that each key (i,k), has list with values (M,j,m_{ij}) and (N, j,n_{jk}) for all possible values of j.

Algorithm for Reduce Function.

d. for each key (i,k) do
e. sort values begin with M by j in listM sort values begin with N by j in listN multiply mij and njk for jth value of each list
f. sum up mij x njk return (i,k), Σj=1 mij x njk

Step 1. Download the hadoop jar files with these links.

Download Hadoop Common Jar files: https://goo.gl/G4MyHp
$ wget https://goo.gl/G4MyHp -O hadoop-common-2.2.0.jar
Download Hadoop Mapreduce Jar File: https://goo.gl/KT8yfB
$ wget https://goo.gl/KT8yfB -O hadoop-mapreduce-client-core-2.7.1.jar

Step 2. Creating Mapper file for Matrix Multiplication.

```
import java.io.DataInput; import
java.io.DataOutput; import
java.io.IOException;
import java.util.ArrayList;

import org.apache.hadoop.conf.Configuration; import
org.apache.hadoop.fs.Path; import
org.apache.hadoop.io.DoubleWritable; import
org.apache.hadoop.io.IntWritable; import
org.apache.hadoop.io.Text; import
org.apache.hadoop.io.Writable; import
org.apache.hadoop.io.WritableComparable; import
org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper; import
org.apache.hadoop.mapreduce.Reducer; import
org.apache.hadoop.mapreduce.lib.input.*; import
org.apache.hadoop.mapreduce.lib.output.*;
import org.apache.hadoop.util.ReflectionUtils;

class Element implements Writable {
      int tag;          int index;       double
value;        Element() {
 tag = 0;   index = 0;
                                value = 0.0;
              }
```

```
                    Element(int tag, int index, double value) {
  this.tag = tag;   this.index = index;
                    this.value = value;
            }
            @Override
      public void readFields(DataInput input) throws IOException {
      tag = input.readInt();          index = input.readInt();
                                      value = input.readDouble();
            }
            @Override
      public void write(DataOutput output) throws IOException {
            output.writeInt(tag);           output.writeInt(index);
                                      output.writeDouble(value);
            }
 }
 class Pair implements WritableComparable<Pair> {
      int i;    int j;

      Pair() {
i = 0;
j = 0;
 }
 Pair(int i, int j) {
            this.i = i;
      this.j = j;
            }
            @Override
      public void readFields(DataInput input) throws IOException {
      i = input.readInt();            j = input.readInt();
            }
            @Override
      public void write(DataOutput output) throws IOException {
            output.writeInt(i);             output.writeInt(j);
            }
            @Override
                  public int compareTo(Pair compare) {
                    if (i > compare.i) {
                    return 1;
                          } else if ( i < compare.i) {
                  return -1;              }
 else {                   if(j > compare.j)
 {
```

```
                                        return 1;
                        } else if (j < compare.j) {
                return -1;
                                }
                        }
                        return 0;
                }
                public String toString() {
                                return i + " " + j + " ";
                }
}
public class Multiply {
public static class MatriceMapperM extends Mapper<Object,Text,IntWritable,Element>
{
@Override
public void map(Object key, Text value, Context context)
  throws IOException, InterruptedException {  String readLine = value.toString();
 String[] stringTokens = readLine.split(",");

 int index = Integer.parseInt(stringTokens[0]);
 double elementValue = Double.parseDouble(stringTokens[2]);
 Element e = new Element(0, index, elementValue);
 IntWritable keyValue = new
IntWritable(Integer.parseInt(stringTokens[1]));
 context.write(keyValue, e);
                        }
                }
 public static class MatriceMapperN extends Mapper<Object,Text,IntWritable,Element> {
                                @Override
 public void map(Object key, Text value, Context context) throws IOException, InterruptedException {
                                                String readLine = value.toString();
                        String[] stringTokens = readLine.split(",");
      int index = Integer.parseInt(stringTokens[1]);
double elementValue = Double.parseDouble(stringTokens[2]);
Element e = new Element(1,index, elementValue);
 IntWritable keyValue = new
IntWritable(Integer.parseInt(stringTokens[0]));
                  context.write(keyValue, e);
                        }
                }
 public static class ReducerMxN extends Reducer<IntWritable,Element, Pair,
DoubleWritable> {
```

```
        @Override
 public void reduce(IntWritable key, Iterable<Element> values, Context context) throws
IOException, InterruptedException {
                                        ArrayList<Element> M = new ArrayList<Element>();
        ArrayList<Element> N = new ArrayList<Element>();
    Configuration conf = context.getConfiguration();
                                    for(Element element : values) {
        Element tempElement = ReflectionUtils.newInstance(Element.class, conf);
                                ReflectionUtils.copy(conf, element, tempElement);

          if (tempElement.tag == 0) {
                M.add(tempElement);
                                        } else if(tempElement.tag == 1) {
                                        N.add(tempElement);
                                }
                        }
                                    for(int i=0;i<M.size();i++) {
                                        for(int j=0;j<N.size();j++) {

     Pair p = new Pair(M.get(i).index,N.get(j).index);
double multiplyOutput = M.get(i).value * N.get(j).value;

                        context.write(p, new DoubleWritable(multiplyOutput));
                                    }
                                }
                        }
        }
                                        public static class MapMxN extends Mapper<Object, Text, Pair,
DoubleWritable> {
                                @Override
                                                public void map(Object key, Text value, Context context)
                                                        throws IOException, InterruptedException {
                                        String readLine = value.toString();
                                        String[] pairValue = readLine.split(" ");
                                    Pair p = new
Pair(Integer.parseInt(pairValue[0]),Integer.parseInt(pairValue[1]));
                                        DoubleWritable val = new
DoubleWritable(Double.parseDouble(pairValue[2]));
                                    context.write(p, val);
                        }
        }
 public static class ReduceMxN extends Reducer<Pair, DoubleWritable, Pair,
```

```
DoubleWritable> {
@Override
  public void reduce(Pair key, Iterable<DoubleWritable> values, Context context) throws
IOException, InterruptedException {
                                double sum = 0.0;
                                        for(DoubleWritable value : values) {
                        sum += value.get();
                  }
                  context.write(key, new DoubleWritable(sum));
}
                }
      public static void main(String[] args) throws Exception {
Job job = Job.getInstance();
      job.setJobName("MapIntermediate");
      job.setJarByClass(Project1.class);
MultipleInputs.addInputPath(job, new Path(args[0]), TextInputFormat.class,
MatriceMapperM.class);
MultipleInputs.addInputPath(job, new Path(args[1]), TextInputFormat.class,
MatriceMapperN.class);
                job.setReducerClass(ReducerMxN.class);
      job.setMapOutputKeyClass(IntWritable.class);
      job.setMapOutputValueClass(Element.class);
      job.setOutputKeyClass(Pair.class);
      job.setOutputValueClass(DoubleWritable.class);
      job.setOutputFormatClass(TextOutputFormat.class);
      FileOutputFormat.setOutputPath(job, new Path(args[2]));
                job.waitForCompletion(true);
      Job job2 = Job.getInstance();
      job2.setJobName("MapFinalOutput");
        job2.setJarByClass(Project1.class);

        job2.setMapperClass(MapMxN.class);
        job2.setReducerClass(ReduceMxN.class);

                job2.setMapOutputKeyClass(Pair.class);
      job2.setMapOutputValueClass(DoubleWritable.class);

                job2.setOutputKeyClass(Pair.class);
      job2.setOutputValueClass(DoubleWritable.class);

        job2.setInputFormatClass(TextInputFormat.class);
        job2.setOutputFormatClass(TextOutputFormat.class);
```

```
        FileInputFormat.setInputPaths(job2, new Path(args[2]));
      FileOutputFormat.setOutputPath(job2, new Path(args[3])); job2.waitForCompletion(true); }
}
```

Step 5. Compiling the program in particular folder named as operation

```
#!/bin/bash

rm -rf multiply.jar classes

module load hadoop/2.6.0

mkdir -p classes javac -d classes -cp classes:`$HADOOP_HOME/bin/hadoop classpath`
Multiply.java jar cf multiply.jar -C classes .

echo "end"
```

Step 6. Running the program in particular folder named as operation

```
export HADOOP_CONF_DIR=/home/$USER/cometcluster module
load hadoop/2.6.0
myhadoop-configure.sh
start-dfs.sh start-yarn.sh

hdfs dfs -mkdir -p /user/$USER hdfs dfs -put M-matrix-large.txt
/user/$USER/M-matrix-large.txt hdfs dfs -put N-matrix-large.txt
/user/$USER/N-matrix-large.txt
hadoop jar multiply.jar edu.uta.cse6331.Multiply /user/$USER/M-matrix-
large.txt
/user/$USER/N-matrix-large.txt /user/$USER/intermediate /user/$USER/output rm
-rf output-distr mkdir output-distr
hdfs dfs -get /user/$USER/output/part* output-distr

stop-yarn.sh
stop-dfs.sh myhadoop-cleanup.sh
```

Expected Output:

```
module load hadoop/2.6.0 rm -rf

output intermediate

hadoop --config $HOME jar multiply.jar edu.uta.cse6331.Multiply M-matrix-small.txt N-matrixsmall.txt
intermediate output
```